

SATNET EDU / Lab 01

Experiments with SatNet Edu

Build a network, record its behavior, and explain a controlled change.

LAB 01

A guided Vancouver–Tokyo experiment

Write Python. Inspect the recording. Compare the outcomes.

Instructor: Yi Ching (David) Chou

1. Install the Simulator in One Python Environment

Use Git, Python 3.11, a text editor, and a browser. Run the commands from your working folder.

- ▶ **Get the repository and enter it.**
 - Clone the project once. All later lab commands run from the folder containing the project README.
- ▶ **Keep installation and execution together.**
 - Create a virtual environment. The variable `$py` selects that environment for every Python command below.
- ▶ **Check that the package imports.**
 - The final command should print OK. If it fails, check the interpreter and installation output before continuing.

Windows PowerShell setup

```
git clone `
  https://github.com/de2008de/satnet-edu.git
cd satnet-edu
py -3.11 -m venv .venv
$py = ".\venv\Scripts\python.exe"
& $py -m pip install .
& $py -c "import satnet_edu; print('OK')"
```

Use the lab README for macOS/Linux commands.
No Docker or LaTeX is needed to run the simulator.

Keep this PowerShell window open so `$py` remains defined. Internet is needed for cloning and installation.

2. Build a Constellation with a Fixed Start Time

Follow the numbered sections in `teaching/lab-01/experiment.py` from the repository root.

► Create one network object.

- Give the experiment a name and a fixed UTC epoch. Equal elapsed times then refer to equal dates in both runs.

► Specify the constellation.

- Six orbital planes with twelve satellites each give 72 satellites. Altitude is in kilometers and inclination in degrees.

► Identify what is still missing.

- The network has satellites, but no ground endpoints yet. The next step adds the two sites and link rules.

Section 1: initialize the orbit configuration

```
from satnet_edu import Network, RouteQuery

net = Network(
    "Lab 01",
    epoch="2026-01-01T00:00:00Z")
net.add_constellation(
    planes=6, sats_per_plane=12,
    altitude_km=1000,
    inclination_deg=53,
)
```

Checkpoint: $6 \times 12 = 72$ satellites, one fixed epoch.

The supplied script is complete. Work through its sections, then execute the file from the repository root.

3. Add Ground Sites and Define Usable Links

Connect the same constellation to Vancouver and Tokyo using stable endpoint IDs A and B.

► **Give each endpoint a stable ID.**

- A and B identify the endpoints in code. City labels make the map easier to read. West longitude is negative.

► **Select candidate space links.**

- The orbital-neighbor policy considers neighboring satellites. A candidate still needs clear geometry and an allowed range.

► **Set the access and range limits.**

- The 10° mask filters ground access. The 4000 km limit filters only inter-satellite links.

Section 2: endpoints and link policy

```
net.add_ground_station(  
    "A", lat=49.28, lon=-123.12,  
    label="Vancouver")  
net.add_ground_station(  
    "B", lat=35.68, lon=139.69,  
    label="Tokyo")  
net.set_link_policy(  
    topology="orbital_neighbors",  
    max_isl_km=4000,  
    min_elevation_deg=10,  
)
```

Ground stations are route endpoints in this model. No terrestrial connection between the two cities is added.

4. Inspect One Route Before Recording a Run

Query a single snapshot at 120 seconds to check that the scenario can answer your question.

► **Ask a specific routing question.**

- Find the minimum-propagation path from A to B at one fixed time. Snapshot queries do not advance a live clock.

► **Check reachability first.**

- Use the path, hop count, and delay only when `reachable` is true. Otherwise inspect the reported reason.

► **Interpret the units correctly.**

- The delay is one-way propagation in milliseconds. It excludes queueing, packet transmission, processing, and server work.

Reference values use the supplied fixed scenario. No path is a valid result for a different configuration.

Section 3: query and inspect

```
route = net.at(120).route(
    "A", "B", metric="delay")
if route.reachable:
    print(route.path)
    print(route.hops)
    print(route.propagation_ms)
else:
    print(route.reason)
```

Reference run: 4 links and about 35.307 ms.
Path: A → S05-04 → S04-04 → S04-03 → B.

5. Record the Baseline and Export a Viewer

A recording stores the network states and route answers that the browser will later replay.

► Record the query you need.

- RouteQuery gives the A-to-B query the name delay. Omitting recorded queries leaves the viewer without route results.

► Choose duration and sample interval.

- The lab records 1200 seconds every 10 seconds. Including both endpoints gives 121 samples.

► Save both data and presentation.

- JSON preserves the recording. The standalone HTML viewer opens in a browser after Python has finished.

Section 4: reusable recording function

```
from pathlib import Path
def record(network, label):
    run = network.run(
        duration_s=1200, step_s=10,
        record_routes=[
            RouteQuery("delay", "A", "B")])
    folder = Path("outputs/lab-01")
    run.save(folder / f"{label}.json")
    run.export_html(folder / f"{label}.html")
    return run
baseline = record(net, "baseline")
```

This script exports English viewers. Playback speed changes replay speed, not the simulation sample interval.

6. Run the File and Inspect Recorded Samples

Execute the supplied script, then open `outputs/lab-01/baseline.html` in your browser.

► Run the complete experiment.

- In PowerShell use `& $py teaching/lab-01/experiment.py`. This produces both recordings.

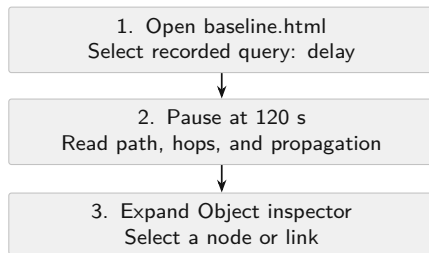
► Pause and choose a recorded time.

- Use Play, Pause, and Next sample. Select the delay query and inspect the path, link hops, and propagation value.

► Inspect the objects on a route.

- Open Object inspector and select a satellite or link. Use link distances to explain the propagation delay.

A repeatable viewer inspection sequence



The browser replays data. Edit Python to change the network.

UI labels match the current English player. The map projects satellite subpoints onto longitude and latitude.

7. Change Only the Failure Schedule

Create a second scenario from the baseline and disable one satellite during a known interval.

- ▶ **Choose a node using a stated rule.**
 - At 600 seconds, select the middle node on the baseline route. Check that a route exists before indexing its path.
- ▶ **Clone instead of rebuilding by hand.**
 - Copy the complete scenario so the epoch, constellation, endpoint locations, and link policy stay identical.
- ▶ **Keep the recording window fixed.**
 - Disable the chosen satellite from 600 s up to, but not including, 900 s. Reuse the same recording function.

Section 5: targeted satellite outage

```
r = net.at(600).route("A", "B")
if not r.reachable or len(r.path) < 3:
    raise RuntimeError("Need a route")
failed_id = r.path[len(r.path) // 2]
failed = Network.from_dict(net.to_dict())
failed.add_failure(
    node_id=failed_id,
    start_s=600, end_s=900)
failure = record(failed, "failure")
```

Reference selection: S04-03.

Open failure.html next to baseline.html at the same time.

The node is disabled on [600,900). This is a targeted stress case, not a random reliability estimate.

8. Compare Reachability and Delay Separately

Read `summary.json` and `paired.csv`, then connect the numerical outcomes to the recorded paths.

► **Count all samples for reachability.**

- Divide reachable samples by all recorded samples. Keep unavailable samples in this denominator.

► **Average delay over reachable samples.**

- A no-path result has no valid delay. Never replace it with zero before calculating a mean.

► **Compare paths at matched times.**

- Use the CSV and both viewers at 600, 700, and 900 s. Explain a changed path, an unchanged path, or a disconnection.

Section 6: reachability and conditional delay

```
frames = baseline.to_dict()["frames"]
routes = [f["routes"][0] for f in frames]
valid = [r for r in routes
         if r["status"] == "reachable"]
fraction = len(valid) / len(routes)
mean_ms = (
    sum(r["propagation_ms"] for r in valid)
    / len(valid) if valid else None)
```

Case	Reachable	Mean delay
Baseline	121/121	35.584 ms
Failure	121/121	35.805 ms

Table: checked reference run. Delay means one-way propagation, conditional on a route. Sample fractions are not continuous availability.

9. Submit a Reproducible Experimental Report

A successful lab result shows what changed, why it changed, and what the model can establish.

► **Preserve a reproducible experiment.**

- Submit your script, both JSON recordings, and both HTML viewers.
State the failure ID and interval.

► **Explain one matched observation.**

- Compare the baseline and failure paths at the same time. Relate the changed links to reachability and propagation delay.

► **Check one limitation experimentally.**

- Save the first outputs. Change only the step from 10 s to 5 s and rerun.
Compare the two sets of metrics.

A short report with concrete checkpoints

Prediction

Will the route disconnect or find a detour?

Observation

Two paths at one time
+ a metric comparison

Explanation

Which link changes account for the result?

State the sample interval. Do not claim throughput or complete application latency.

A 5-second step gives 241 samples over 1200 seconds. Keep the 10-second outputs in a separate folder before rerunning.